

Projet UL2IN002 – 2026fev Thème : “la Mer”

Numéro de groupe :9

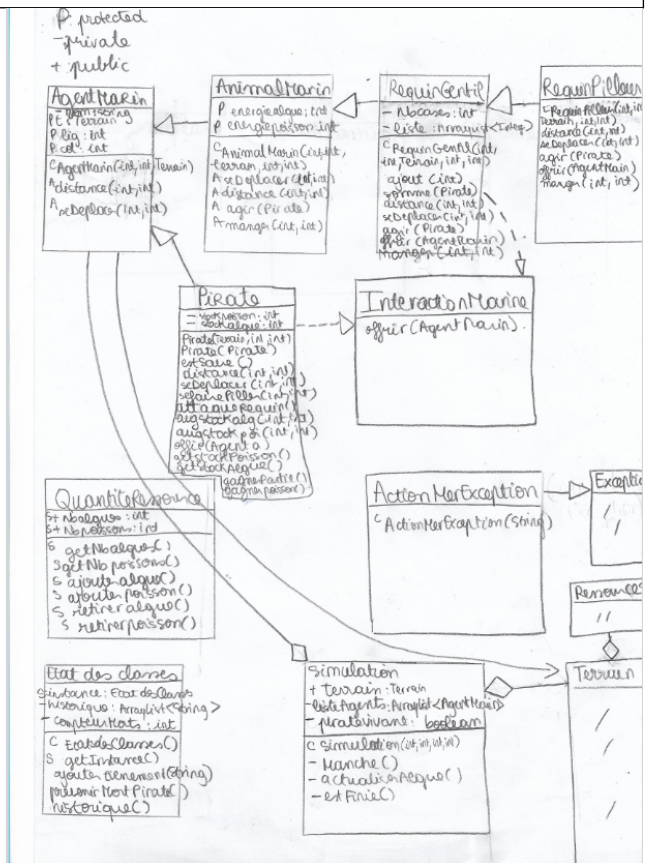
Nom :Stoj	Nom :
Prénom :Laura	Prénom :
N° étudiant :21413129	N° étudiant :

	Thème de la simulation
<i>Thème et objectif(s) de votre simulation : (5 lignes maximum)</i>	Il ya 2 ressources: algues et poissons magiques et des agents marins :pirate et animaux marins : requin gentil, pilleur, le requin gentil offre un poisson au pirate dans 60% des cas quand c'est son tour, le requin pilleur tue ou vide le stock du pirate, le pirate peut aussi augmenter son stock si il tombe sur la bonne case: il gagne si il a 3 poissons magiques.
<i>Les agents :</i>	Pirate, Requiringentil, RequinPilleur
<i>Les ressources :</i>	Algue, poisson magique

Quel est le nombre de classes/interfaces de votre projet ? (ne pas compter Ressource, Terrain et TestTerrain)	11 classes
Avez-vous une hiérarchie d'héritage à 3 niveaux ?	AgentMarin, AnimalMarin, Requiringentil,RequinPilleur
Avez-vous généré la javadoc ?	Oui
Avez-vous généré des logs ?	Oui

Checklist des points vus en cours à mettre en application :	Nom de classe correspondant (une seule classe demandée, mais vous pouvez en mettre plus)
Classe contenant une ArrayList	Simulation
Classe abstraite et méthode abstraite	AgentMarin, AnimalMarin
Interface	InteractionMarine
Classe avec un constructeur par copie ou méthoe clone()	Pirate
Classe contenant que des attributs et méthodes statiques	QuantiteRessource
Classe héritant de Exception	ActionMerException
Gestion des exceptions	Oui dans manger si la case est vide
Singleton ou une de ses variantes	EtatDesClasses

Schéma UML des classes vision fournisseur (dessin “à la main” scanné ou photo acceptés)



Copier-coller de 2 ou 3 logs (affichages de simulations)

LOG 1:

DÉBUT DU JEU
Requin Pilleur se déplace en 6,16
TOUR DU REQUIN PILLEUR
Je préfère manger des algues que piller.
Incident en mer!
TOUR DU PIRATE
Pirate se déplace en 11,10 et a 0 poissons magiques.
POISSONS DU PIRATE: 0
Requin gentil se déplace en 17,12
TOUR DU REQUIN GENTIL
Je t'offre un poisson
J'offre un poisson au pirate!
POISSONS DU PIRATE: 1
TOUR DU PIRATE
Pirate se déplace en 12,3 et a 1 poissons magiques.
POISSONS DU PIRATE: 1
Requin Pilleur se déplace en 2,9
TOUR DU REQUIN PILLEUR
Pirate tué, Félicitations aux requins.
Un mort de plus !
MESSAGE : Il y a eu un mort (pirate).
LE PIRATE EST MORT VICTOIRE DES REQUINS !
résumé :
Il y a eu un mort (pirate).

LOG 2 PARTIE 1 :

DÉBUT DU JEU

Requin Pilleur se déplace en 12,16

TOUR DU REQUIN PILLEUR

Je préfère manger des algues que piller.

Incident en mer!

TOUR DU PIRATE

Pirate se déplace en 3,19 et a 0 poissons magiques.

POISSONS DU PIRATE: 0

Requin gentil se déplace en 2,5

TOUR DU REQUIN GENTIL

Je t'offre un poisson

J'offre un poisson au pirate!

POISSONS DU PIRATE: 1

TOUR DU PIRATE

Pirate se déplace en 20,11 et a 1 poissons magiques.

POISSONS DU PIRATE: 1

Requin Pilleur se déplace en 10,20

TOUR DU REQUIN PILLEUR

Je préfère manger des algues que piller.

Incident en mer!

TOUR DU PIRATE

Pirate se déplace en 9,9 et a 1 poissons magiques.

POISSONS DU PIRATE: 1

Requin gentil se déplace en 20,14

TOUR DU REQUIN GENTIL

Je t'offre un poisson

J'offre un poisson au pirate!

POISSONS DU PIRATE: 2

TOUR DU PIRATE

Pirate se déplace en 5,18 et a 2 poissons magiques.

POISSONS DU PIRATE: 2

LOG 2 SUITE :

```

TOUR DU PIRATE
Pirate se déplace en 5,18 et a 2 poissons magiques.
  POISSONS DU PIRATE: 2
Requin Pilleur se déplace en 6,3
TOUR DU REQUIN PILLEUR
Je préfère manger des algues que piller.
Incident en mer!
  TOUR DU PIRATE
Pirate se déplace en 10,7 et a 2 poissons magiques.
  POISSONS DU PIRATE: 2
Requin gentil se déplace en 8,11
TOUR DU REQUIN GENTIL
Je t'offre un poisson
J'offre un poisson au pirate!
  POISSONS DU PIRATE: 3
  TOUR DU PIRATE
Pirate se déplace en 13,8 et a 3 poissons magiques.
  POISSONS DU PIRATE: 3
LE PIRATE SURVIT ET GAGNE LA PARTIE !

```

Copier-coller vos classes et interfaces à partir d'ici :

```

public abstract class AgentMarin{
    private String nom;
    protected Terrain t;
    protected int lig, col;
    public AgentMarin(int lig, int col, Terrain t){
        this.lig=lig;
        this.col=col;
        this.t=t;
    }
    public abstract double distance(int lig, int col);
    public abstract void seDeplacer(int lig,int col);
}

```

```

public abstract class AnimalMarin extends AgentMarin{
    private String nom;
    protected int energiealgue;
}

```

```

    protected int energiepoisson;
    public AnimalMarin(int energiealgue, int energiepoisson, Terrain terrain, int
lig, int col){
        super(lig, col, terrain);
        this.energiealgue=energiealgue;
        this.energiepoisson=energiepoisson;
    }
    public abstract void seDeplacer(int lig, int col);
    public abstract String getNom();
    public abstract double distance(int lig, int col);
    public abstract boolean agir(Pirate p);
    public abstract void manger(int i, int j) throws ActionMerException;
}

```

```

public class Pirate extends AgentMarin implements InteractionMarine{
    private String nom="Pirate";
    private int stockpoisson;
    private int stockalgue;
    public Pirate( Terrain terrain, int lig, int col){
        super(lig, col, terrain);
        this.stockpoisson=0;
        this.stockalgue=0;
    }
    public Pirate(Pirate autrePirate){
        super(autrePirate.lig, autrePirate.col, autrePirate.t );
        this.stockpoisson=autrePirate.stockpoisson;
        this.stockalgue=autrePirate.stockalgue;
    }
    public boolean estSauve(){
        return stockpoisson>=2;
    }
    public double distance(int lig, int col){
        double dislig=Math.pow ((lig-this.lig),2);
        double discol=Math.pow ((col-this.col),2);
        return Math.sqrt(dislig+discol);
    }
    public void seDeplacer(int lig, int col){
        this.lig=lig;
        this.col=col;
        System.out.println(this.nom +" se déplace en " +lig + "," + col+ " et a
"+stockpoisson+" poissons magiques.");
    }
    public void seFairePiller(int algues, int poissons){
        stockalgue-=algues;
        stockpoisson-=poissons;
        if(stockalgue < 0) stockalgue = 0;
        if(stockpoisson < 0) stockpoisson =0;
    }
    public boolean attaqueRequin(){
        if(!estSauve()){
            System.out.println("Pirate tué, Félicitations aux requins.");
        }
    }
}

```

```

        EtatDesClasses.getInstance().prevenirMortPirate();;
        EtatDesClasses.getInstance().ajouterEvenement("Il y a eu un mort
(pirate).");
        return true;
    }else{
        System.out.println("J'ai assez de poissons magiques et je survie!");
        return false;
    }
}

public void augstockalgue(int i , int j){
    Ressource r = t.getCase(i, j);
    if(r!=null){
        if((i==lig && j==col) && "Algue".equals(r.type)){
            System.out.println("J'ai "+ stockalgue+ " algues!");
            stockalgue++;
            t.viderCase(lig, col);
        }
    }
}

public void augstockpoisson(int i , int j){
    Ressource r = t.getCase(i, j);
    if(r!=null){
        if((i==lig && j==col) || (t.sontValides(i+1, j) && i+1==lig && j==col)
|| (t.sontValides(i, j+1) && i==lig && j+1==col) && "PoissonM".equals(r.type)){
            System.out.println("J'ai "+stockpoisson+ " poissons magiques!");
            stockpoisson++;
            t.viderCase(lig, col);
        }
    }
}

public void gagnerpoisson(){
    stockpoisson++;
}

public void offrir(AgentMarin a){
    if(a instanceof Requiringentil){
        if(stockalgue>=1){
            ((Requiringentil)(a)).energpoisson();
            stockalgue--;
        }
    }else{
        System.out.println("pas assez d'algues");
    }
}

public int getStockPoisson(){
    return stockpoisson;
}

public int getStockAlgue(){
    return stockalgue;
}

public boolean gagnePartie(){
    if(stockpoisson>=3){

```

```

        return true;
    }return false;
}
}

```

```

import java.util.ArrayList;
public class Requiringentil extends AnimalMarin implements InteractionMarine{
    private String nom;
    public Requiringentil(int algue, int poisson, Terrain t, int lig, int col){
        super(algue, poisson, t, lig, col);
        this.nom="Requin gentil";
    }
    public String getNom(){
        return nom;
    }
    public double distance(int lig, int col){
        double dislig=Math.pow ((lig-this.lig),2);
        double discol=Math.pow ((col-this.col),2);
        return Math.sqrt(dislig+discol);
    }
    public void energypoison(){
        energiepoisson++;
    }
    public void seDeplacer(int lig, int col){
        this.lig=lig;
        this.col=col;
        System.out.println(this.nom +" se déplace en " +lig + "," + col);
    }
    public boolean agir(Pirate p, int i , int j) throws ActionMerException{
        energiepoisson++;
        if(Math.random()>0.4){
            offrir(p);
            return true;
        }else{
            manger(i, j);
            return false;
        }
    }
    public void offrir(AgentMarin a){
        if(a instanceof Pirate){
            ((Pirate)a).gagnerpoisson();
            System.out.println("Je t'offre un poisson");
        }
    }
    public void manger(int i , int j) throws ActionMerException{
        if(i==lig && j==col){
            Ressource r = t.getCase(i, j);
            if(r==null)
                throw new ActionMerException("La case n'a aucune ressource!");
            if ("Algue".equals(r.type)) {

```



```

        QuantiteRessource.retirerAlgue();
        energiealgue++;
        System.out.println("REQUIN GENTIL : Je mange une algue");
        System.out.println("Algues en mer :
"+QuantiteRessource.getNbAlgues()+" Poissons en mer :
"+QuantiteRessource.getNbPoissons());
        if ("PoissonM".equals(r.type)){
            QuantiteRessource.retirerPoisson();
            energiepoisson++;
            System.out.println("REQUIN GENTIL : Je mange un poisson");
            System.out.println("Algues en mer :
"+QuantiteRessource.getNbAlgues()+" Poissons en mer :
"+QuantiteRessource.getNbPoissons());
            System.out.println("Je mange l'algue !");
            t.viderCase(lig, col);
        }
    }
}

```

```

public class RequinPilleur extends Requiringentil{
    public RequinPilleur(int algue, int poisson, Terrain t, int lig, int col){
        super(algue, poisson, t, lig, col);
    }
    public String getNom(){
        return "Requin Pilleur";
    }
    public double distance(int lig, int col){
        double dislig=Math.pow ((lig-this.lig),2);
        double discol=Math.pow ((col-this.col),2);
        return Math.sqrt(dislig+discol);
    }
    public void seDeplacer(int lig, int col){
        this.lig=lig;
        this.col=col;
        System.out.println(this.getNom() +" se déplace en " +lig + "," + col);
    }
    public boolean agir(Pirate p, int i, int j) throws ActionMerException{
        double de=Math.random();
        if(de<=0.1){
            p.attaquerRequin();
            return true;
        }else if(de<=0.2){
            p.seFairePiller(3, 3);
            return false;
        }else{
            System.out.println("Je préfère manger des algues que piller.");
            manger(i, j);
            return false;
        }
    }
    public void offrir(AgentMarin a){

```

```

        System.out.println("je ne t offrirai rien je ne suis pas gentil");
    }
    public void manger(int i , int j) throws ActionMerException{
        if(i==lig && j==col){
            Ressource r = t.getCase(i, j);
            if(r==null)
                throw new ActionMerException("La case n'a aucune ressource!");
            if ("Algue".equals(r.type)){
                QuantiteRessource.retirerAlgue();
                System.out.println("REQUIN PILLEUR : Je mange une algue");
                System.out.println("Algues en mer : 
"+QuantiteRessource.getNbAlgues()+" Poissons en mer : 
"+QuantiteRessource.getNbPoissons());
                energiealgue++;
            }if ("PoissonM".equals(r.type)) if(Math.random() >0.9 ){
                QuantiteRessource.retirerPoisson();
                energiepoisson++;
                System.out.println("REQUIN PILLEUR : Je mange un poisson");
                System.out.println("Algues en mer : 
"+QuantiteRessource.getNbAlgues()+" Poissons en mer : 
"+QuantiteRessource.getNbPoissons());
            }
            System.out.println("Je mange l'algue ou le poisson!");
            t.viderCase(lig, col);
        }
    }
}

```

```

public class ActionMerException extends Exception{
    public ActionMerException(String message){
        super(message);
    }
}

```

```

public interface InteractionMarine{
    public void offrir(AgentMarin a);
}

```

```

public class QuantiteRessource{
    public static int nbalgues=0;
    public static int nbpoissons=0;
    public static int getNbAlgues(){
        return nbalgues;
    }
    public static int getNbPoissons(){
        return nbpoissons;
    }
    public static void ajouterAlgue(){
        nbalgues++;
    }
    public static void ajouterPoisson(){

```

```

        nbpoissons++;
    }
    public static void retirerAlgue(){
        if(nbalgues>0) nbalgues--;
    }
    public static void retirerPoisson(){
        if(nbpoissons>0) nbpoissons--;
    }
}

```

```

import java.util.ArrayList;
public class EtatDesClasses {
    private static EtatDesClasses instance = null;
    private ArrayList<String> historique;
    private int compteurMorts;
    private EtatDesClasses() {
        historique = new ArrayList<>();
        compteurMorts= 0;
    }
    public static EtatDesClasses getInstance() {
        if (instance == null) {
            instance = new EtatDesClasses();
        }
        return instance;
    }
    public void ajouterEvenement(String msg) {
        historique.add(msg);
        System.out.println(" MESSAGE : " + msg);
    }

    public void prevenirMortPirate() {
        compteurMorts++;
        System.out.println("Un mort de plus !");
    }
    public void historique() {
        System.out.println("résumé : ");
        for (String s : historique) {
            System.out.println(s);
        }
    }
}

```

```

import java.util.ArrayList;
public class Simulation {
    public Terrain terrain;
    private ArrayList<AgentMarin> listeAgents;
    private boolean pirateVivant = true;

    public Simulation(int nbress, int nbag, int li, int co) {
        this.terrain = new Terrain(li, co);
        this.listeAgents = new ArrayList<AgentMarin>();
    }
}

```

```

int nbr = 0;
for (int i = 1; i <= terrain.nbLignes; i++) {
    for (int j = 1; j <= terrain.nbColonnes; j++) {
        if (nbr < nbress) {
            if (Math.random() > 0.2 && terrain.caseEstVide(i, j)) {
                terrain.setCase(i, j, new Ressource("PoissonM", 1));
                QuantiteRessource.ajouterPoisson();
                nbr++;
            } else if (Math.random() > 0.8 && terrain.caseEstVide(i, j)) {
                terrain.setCase(i, j, new Ressource("Algue", 1));
                QuantiteRessource.ajouterAlgue();
                nbr++;
            }
        }
    }
}

for (int i = 0; i < nbag; i++) {
    int h1 = (int)(Math.random()* li) + 1;
    int h2 = (int)(Math.random()* co) + 1;

    if (i == 0) {
        listeAgents.add(new Pirate( terrain, h1, h2));
    } else {
        if (i%2 == 0) {
            listeAgents.add(new Requiringtil(0, 0, terrain, h1, h2));
        } else {
            listeAgents.add(new RequinPilleur(0, 0, terrain, h1, h2));
        }
    }
}

public void manche() throws ActionMerException{
    if (!pirateVivant) return;
    Pirate lePirate = (Pirate) (listeAgents.get(0));

    for (AgentMarin agent : listeAgents) {
        if (agent == null || agent == lePirate) continue;

        if (agent instanceof AnimalMarin) {
            AnimalMarin am = (AnimalMarin) agent;
            if (am instanceof RequinPilleur) {
                int ml = (int) (Math.random() * terrain.nbLignes) + 1;
                int mc = (int) (Math.random() * terrain.nbColonnes) + 1;
                am.seDeplacer(ml, mc);
                System.out.println("TOUR DU REQUIN PILLEUR");
                try{
                    if (((RequinPilleur)am).agir(lePirate, ml, mc)) {
                        this.pirateVivant = false;
                        System.out.println("LE PIRATE EST MORT VICTOIRE DES
REQUINS !");

```

```

        return;
    } else {
        System.out.println("LE PIRATE SE FAIT PILLER DE 3 POISSONS
ET 3 ALGUES !");
        System.out.println(" POISSONS DU PIRATE:
"+lePirate.getStockPoisson());
    }
    }catch(ActionMerException e){
        System.out.println("Incident en mer!");
    }
}

} else if (am instanceof Requiringentil) {
    int ll = (int) (Math.random() * terrain.nbLignes) + 1;
    int lc = (int) (Math.random() * terrain.nbColonnes) + 1;
    am.seDeplacer(ll, lc);
    System.out.println("TOUR DU REQUIN GENTIL");
    try{
        if(((Requiringentil)am).agir(lePirate, ll, lc)){
            System.out.println("J'offre un poisson au pirate!");
            System.out.println(" POISSONS DU PIRATE:
"+lePirate.getStockPoisson());
        }else{
            System.out.println("Je me balade et je n'offre rien");
        }
    }catch(ActionMerException e){
        System.out.println("Incident en mer!");
    }
}

}

System.out.println(" TOUR DU PIRATE ");
int nl = (int) (Math.random() * terrain.nbLignes) + 1;
int nc = (int) (Math.random() * terrain.nbColonnes) + 1;
lePirate.seDeplacer(nl, nc);
lePirate.augstockalgue(nl, nc);
lePirate.augstockpoisson(nl, nc);
System.out.println(" POISSONS DU PIRATE: " + lePirate.getStockPoisson());

if (lePirate.gagnePartie()) {
    System.out.println("LE PIRATE SURVIT ET GAGNE LA PARTIE !");
}
}
actualiserAlgues();
}

private void actualiserAlgues() {
    for (int i = 1; i <= terrain.nbLignes; i++) {
        for (int j = 1; j <= terrain.nbColonnes; j++) {
            Ressource r = terrain.getCase(i, j);
            if (r != null && "Algue".equals(r.type)) {
                r.setQuantite(r.getQuantite() + 1);
                QuantiteRessource.ajouterAlgue();
            }
        }
    }
}

```

```

        }
    }
}
public boolean estFinie() {
    if (!pirateVivant) return true;
    Pirate p = (Pirate)(listeAgents.get(0));
    return p.gagnePartie();
}
}

```

```

public class TestSimulation {
    public static void main(String[] args) {
        Simulation sim = new Simulation(10, 3, 20, 20);
        System.out.println(" DÉBUT DU JEU ");
        int nb=0;
        while (!sim.estFinie() && nb<15) {
            nb++;
            try {
                sim.manche();
            } catch (ActionMerException e) {
                System.out.println("Un incident est survenu en mer");
            }
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {}
        } if(nb==15) System.out.println("Trop de manches!!!");
        EtatDesClasses.getInstance().historique();
    }
}

```